CWM-FDI Work Journal Assignment 1

Question 1:

-Run time of matmul_slow.py

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
```

-Time results of the program

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

real    0m0.233s
user    0m0.224s
sys     0m0.009s
```

\

-Time measured using /usr/bin/time

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ /usr/bin/time -v time python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
0.23user 0.01system 0:00.25elapsed 99%CPU (0avgtext+0avgdata 13276maxresident)k
164inputs+0outputs (0major+1946minor)pagefaults 0swaps
        Command being timed: "time python3 matmul_slow.py"
        User time (seconds): 0.23
        System time (seconds): 0.01
        Percent of CPU this job got: 98%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:00.25
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 13276
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 2032
        Voluntary context switches: 47
        Involuntary context switches: 13
        Swaps: 0
        File system inputs: 232
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

Difference between real, user, and sys?

A: The real time is the total time used to run this program, which is the sum of user time and sys time

Q2:

Which summary statistics would you report?

I will report the mean value of the 5 run times. This is because later as we use this program multiple times, we would like to know its average performance.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ for i in 1 2 3 4 5; do /usr/bin/time -f "%e" python3 matmul_slow.py; d
one
n=128 reps=2 checksum=107389.290000
0.23
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.22
n=128 reps=2 checksum=107389.290000
0.23
```

Q3:

The output shows the number of cycles and instructions executed by the program.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -e cycles,instructions python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000

 Performance counter stats for 'python3 matmul_slow.py':

         804174757      cycles
        2299495372      instructions                     #    2.86  insn per cycle

       0.226327065 seconds time elapsed

       0.217545000 seconds user
       0.007983000 seconds sys
```

Q4:

The instruction to cycle ratio of this program is 2.82 insn per cycle. This suggests that in one cycle, at least 2 instructions are executed. The cache-misses observed is 581938. It matters because it reflects the efficiency and speed of running the program. Utilization of more cache resources will lead to shorter run time and less usage of CPU.

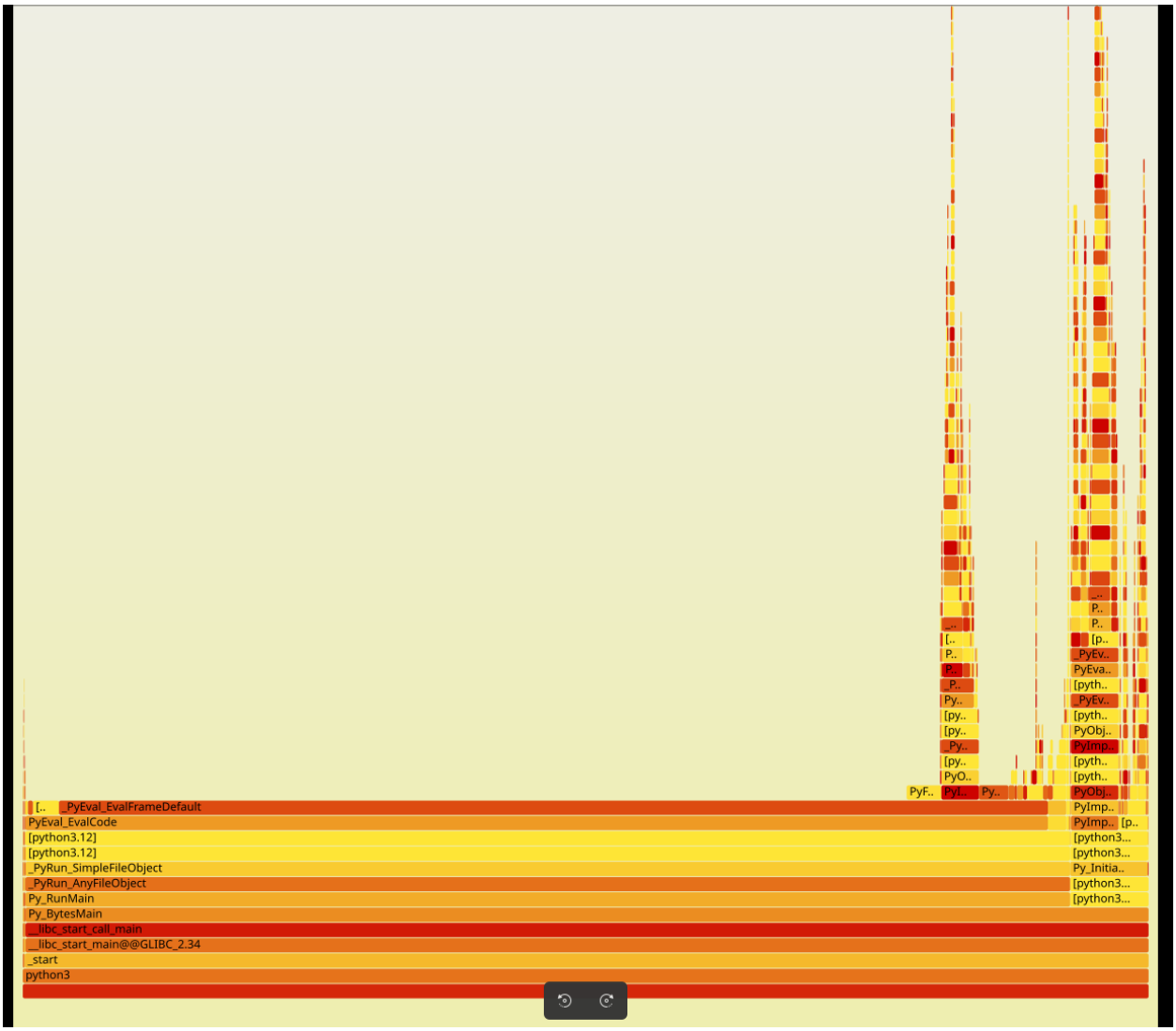
```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

 Performance counter stats for 'python3 matmul_slow.py' (5 runs):

         812695123      cycles                                                  ( +-  0.98% )
        2295376454      instructions              #    2.82  insn per cycle     ( +-  0.04% )
            581938      cache-misses                                            ( +-  3.79% )

          0.22746 +- 0.00216 seconds time elapsed  ( +-  0.95% )
```

Q5:

The result shows that the PyEval_EvalFrameDefault function is consuming the largest fraction of samples. This reveals that the python is running a loop so many times in the program.

```
Samples: 929  of event 'cycles:P', Event count (approx.): 816425628
  Children      Self  Command  Shared Object          Symbol
+   99.81%     0.00%  python3  python3.12             [.] _start
+   99.81%     0.00%  python3  libc.so.6              [.] __libc_start_main@@GLIBC_2.34
+   99.81%     0.00%  python3  libc.so.6              [.] __libc_start_call_main
+   99.81%     0.00%  python3  python3.12             [.] Py_BytesMain
+   95.83%     0.00%  python3  python3.12             [.] PyEval_EvalCode
+   94.16%    76.68%  python3  python3.12             [.] _PyEval_EvalFrameDefault
+   92.95%     0.00%  python3  python3.12             [.] Py_RunMain
+   92.95%     0.00%  python3  python3.12             [.] _PyRun_AnyFileObject
+   92.95%     0.00%  python3  python3.12             [.] _PyRun_SimpleFileObject
+   91.11%     0.00%  python3  python3.12             [.] 0x00000000006b5263
+   90.89%     0.00%  python3  python3.12             [.] 0x0000000000608b52
+    8.44%     0.00%  python3  python3.12             [.] PyImport_ImportModuleLevelObject
+    8.44%     0.00%  python3  python3.12             [.] PyObject_CallMethodObjArgs
+    8.44%     0.00%  python3  python3.12             [.] 0x000000000054a087
+    8.01%     0.00%  python3  python3.12             [.] 0x000000000058206d
+    7.60%     0.00%  python3  python3.12             [.] 0x00000000005d36ac
+    6.86%     0.00%  python3  python3.12             [.] 0x00000000006bc965
+    6.74%     0.00%  python3  python3.12             [.] 0x00000000006bca35
+    6.74%     0.00%  python3  python3.12             [.] Py_InitializeFromConfig
+    5.55%     0.00%  python3  python3.12             [.] 0x00000000006b0bc4
+    5.46%     0.00%  python3  python3.12             [.] 0x00000000005d39f4
+    5.01%     3.70%  python3  python3.12             [.] PyFloat_FromDouble
+    4.93%     0.00%  python3  python3.12             [.] PyImport_ImportModule
+    4.93%     0.00%  python3  python3.12             [.] PyImport_Import
+    4.93%     0.00%  python3  python3.12             [.] PyObject_CallFunction
+    4.59%     0.10%  python3  python3.12             [.] PyObject_Vectorcall
+    4.03%     0.00%  python3  python3.12             [.] 0x00000000006b1132
+    3.02%     0.10%  python3  python3.12             [.] _PyObject_MakeTpCall
```

Q6:



From this graph, the vertical direction shows the order or layer of commands. The difference in the width of each block will show that how long does it take for the command to operate. From this graph, it shows that the function PyEval_EvalFrameDefault takes the longest time to execute as its children are small. Therefore, in this sense, the bottlenecks is that function in the *matmul_slow.py*.

Q7:

To optimize the slow multiplication, implementing and testing 3 faster methods from matmul_fast.

For method 1, it reorders loops to improve the locality, thus improving in the usage of cache. The result shows that there is a significant reduction in the cycles run, but an increase in the cache missed.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

 Performance counter stats for 'python3 matmul_slow.py' (5 runs):

        735118005      cycles                                                        ( +-  0.27% )
       2058757566      instructions              #    2.80  insn per cycle           ( +-  0.05% )
           833426      cache-misses                                                  ( +-  9.20% )

          0.20642 +- 0.00102 seconds time elapsed  ( +-  0.49% )
```

Time spent now is 0.20642 in average.

For method 2, the loop is reordered to reduce the number of inner loops. This will decrease the total number of loops executed by the program.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -r 5 -e cycles,instructions,cache-misses python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000
n=128 reps=2 checksum=107389.290000

 Performance counter stats for 'python3 matmul_slow.py' (5 runs):

        753264176      cycles                                                        ( +-  0.12% )
       2272170957      instructions              #    3.02  insn per cycle           ( +-  0.04% )
           616453      cache-misses                                                  ( +-  3.10% )

          0.21251 +- 0.00199 seconds time elapsed  ( +-  0.94% )
```

From the result, it shows that an obvious decrease in the number of cycles run, and less cache is missed than method 1. However, the run time of method 2 is longer than method 1.

For method 3, it transposes matrix B to largely reduce the number of cycles run and get a significant decrease in run time. This is due to the fact that when the program is accessing the row of a matrix, it utilizes spatial locality better than accessing the column of the matrix.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ sudo perf stat -e cycles,instructions,cache-misses python3 matmul_fast.py
n=128 reps=2 checksum=107389.290000

 Performance counter stats for 'python3 matmul_fast.py':

        640189967      cycles
       1840708880      instructions              #    2.88  insn per cycle
          726948      cache-misses

      0.183159291 seconds time elapsed

      0.178145000 seconds user
      0.004003000 seconds sys


ubuntu@ubuntu:~/CWM-FDI/assignment1$
```